WriteUp

0X00

首先安装这个应用看看怎么个事吧。

好吧，没发现什么东西，用 jadx 打开这个 apk 看看，然后看看 com.example.angler.MainActivity 这是安卓应用的入口点函数，看看这里写了什么东西。

```
public final void onCreate(Bundle bundle) {
    super.onCreate(bundle);
    setContentView(R.layout.activity_main);
    this.v = (TextView) findViewById(R.id.textView2);
    this.f1753w = (TextView) findViewById(R.id.textView);
    this.f1754x = (ImageView) findViewById(R.id.imageView);
    registerReceiver(this.f1756z, new IntentFilter("android.intent.action.BATTERY_LOW"));
}
```

发现它重写了 onCreate 函数，这里的逻辑，定义了几个 class 里的变量，为文本框和图像框，注册了一个 reveiver 函数，传递了一个安卓消息，

android.intent.action.BATTERY_LOW

又观察看到 a 继承了 BroadcasstReceiver 函数，重写了 onReceive 函数，然后它在这个函数里通过 intent 的消息来走执行路径，因为 else 看到了就是现在什么也没做的时候，手机 screen 显示的文本。让我们发一个消息看看会变成什么样。

```
if (intent.getStringExtra("Is_on").equals("yes")) {
```

am broadcast -a “android.intent.action.BATTERY_LOW” - - es “Is_on” “yes”

am 是活动管理器 - - es 是附带参数，可以看到它们判断是通过一个 Is_on:yes 的键值对进行判断的。发了这个消息之后就可以看到

感觉没什么屌用。

0X10

.

```
                Toast.makeText(context, "Look me inside", 1).show();
                printStream = System.out;
                info = MainActivity.this.getInfo(d.d("XDR"));
            } else {
                printStream = System.out;
                info = "I am Strong, no one can defeat me";
            }
            printStream.println(info);
        }
    }

    static {
        System.loadLibrary("angler");
    }
```

发现它把输出重定向到 printStream 里了，让我们看看 system.out 里面有什么

```
03-17 21:57:40.165 11443 11443 I System.out: I am not here, I am there
03-17 21:57:40.197 11443 11443 I System.out: I am not here, I am there
03-17 21:57:40.240 11443 11443 I System.out: I am not here, I am there
03-17 22:19:37.780 11443 11443 I System.out: I am not here, I am there
03-17 22:19:37.801 11443 11443 I System.out: I am not here, I am there
03-17 22:19:37.830 11443 11443 I System.out: I am not here, I am there
03-17 22:19:37.840 11443 11443 I System.out: I am not here, I am there
```

额，发现还是不在这，我了个骚肛。

```
                printStream = System.out;
                info = MainActivity.this.getInfo(d.d("XDR"))
            } else {
                printStream = System.out;
                info = "I am Strong, no one can defeat me";
            }
            printStream.println(info);
        }
    }

    static {
        System.loadLibrary("angler");
    }

    public native String getInfo(String str);
```

发现它用 loadLibrary 加载了一个 angler 的库，并且还调用了，那我们看看这个库到底

是怎么回事吧。

0x20

用 apk -d angler.apk 解压这个 apk 包，然后用 ida 打开这个 libangler.so 的库

```
  if ( !strcmp(s1, v23) )
    std::string::basic_string<decltype(nullptr)>((int)a1, "You found the flag");
  else
    std::string::basic_string<decltype(nullptr)>((int)a1, "I am not here, I am there");
  if ( (v28 & 1) != 0 )
    operator delete(v22);
  if ( (v26 & 1) != 0 )
```

通过 string 先搜索一下 flag，然后定位到这个地方。这里有一个 strcmp 的函数，应该就是对比某个string 和真实的 flag 值。感觉这个 strcmp 其中有一个真实的 flag 值，通过这个函数回溯一下，发现它走到了

```
int __cdecl Java_com_example_angler_MainActivity_getInfo(int a1, int a2, int a3)
{
  char *v3; // esi
  _BYTE *v4; // esi
  char v6; // [esp+18h] [ebp-24h] BYREF
  _BYTE v7[3]; // [esp+19h] [ebp-23h] BYREF
  _BYTE *v8; // [esp+20h] [ebp-1Ch]
  unsigned int v9; // [esp+28h] [ebp-14h]

  v9 = __readgsdword(0x14u);
  v3 = (char *)(*(int (__cdecl **)(int, int, _DWORD))(*(_DWORD *)a1 + 676))(a1, a3, 0);
  illusion(v3);
  ne(&v6, v3);
  if ( (v6 & 1) != 0 )
  {
    v4 = v8;
    operator delete(v8);
  }
  else
  {
    v4 = v7;
```

定义的那个静态函数里，是通过 ne 传递调用下去的，发现这里的 v6 和 v3 都不是很好静态分析出来，因为它们都是动态计算处理啊的，这里还有几个虚函数，感觉静态分析好麻烦啊，感觉干脆直接上 frida 来 hook 这个 strcmp，然后把真实的 flag 打印出来即可。从经验分析，一般真实的值在 strcmp(arg0,arg1)中是 arg1.

行把，写一个 frida 脚本吧。笔者在这里配环境配了半天，报缺少 libart.so 的库，通过

export LD_LIBRARY_PATH=/apex/com.android.runtime/lib64:/apex/com.android.runtime/lib64:/system/lib64:/system/lib64:/data/local/tmp

就可以把 frida-server 启动起来了。

0x30

```
└─$ cat -n  hook_angler.js
     1  const flagliba = Process.getModuleByName("libc.so");
     2  const getStrcmp = flagliba.getExportByName('strcmp');
     3
     4  Interceptor.attach(getStrcmp,{
     5          onEnter(args){
     6
     7
     8
     9                  if(!args[1].isNull()){
    10                  var flag = args[1].readUtf8String();
    11                  console.log("flag: "+flag);
    12                  }
    13
    14
    15  }
    16
    17  });
```

简单写了一个脚本，没有加过滤，有兴趣的读者可以通过 backtraces 加上一个 module 的过滤就行呢。

```
flag: miui_optimization
flag: miui_optimization
flag: miui_optimization
flag: miui_optimization
flag: miui_optimization
flag: miui_optimization
flag: 4854427b796f755f3472335f676f6f645f34745f6830306b316e397d
flag: TK
flag: MS
flag: I
flag: Landroid/os/Parcel;
flag: Landroid/os/Parcel;
flag: I
flag: [B
flag: B
flag: [F
```

观察到它的输出很多干扰项，但是有经验的读者一眼就可以看出来 ascii 的值范围，这个很长的串，应该就是 flag 的 ascii 值了。估计 frida 的.readUtf8String 只是读出'\0'结尾前的所有数据吧，不过也很奇怪啊，这里应该都是 char 了，为什么显示的是一串 ascii 码值？应该是故意的，防止我们直接搜索出来这个 string。 最后一步了。

```
└─$ echo \4854427b796f755f3472335f676f6f645f34745f6830306b316e397d | xxd -r -p
HTB{you_4r3_good_4t_h00k1n9}
```

Ok